Project Executable Files

Date	1 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
SmartLender/
├── Dataset/
│   ├── loan_prediction.csv          # Raw loan dataset (CSV format)
│   └── loan_prediction.xlsx         # Raw loan dataset (Excel format)
│
├── Flask/
│   ├── app.py                       # Core Flask application, validation logic, & inference endpoint
│   ├── rdf.pkl                      # Trained and serialized XGBoost Classifier
│   ├── scale.pkl                    # Serialized StandardScaler object for normalizing input data
│   │
│   ├── templates/                   # HTML structure for the web interface
│   │   ├── home.html                # Landing page introducing the system features
│   │   ├── predict.html             # Dynamic loan prediction form with real-time UI widgets
│   │   └── submit.html              # Display dashboard for approval status, risk band, and confidence
│   │
│   └── static/                      # Static visual assets and client-side logic
│       ├── css/
│       │   └── style.css            # Custom stylesheets containing light/dark theme variables
│       └── js/
│           └── script.js            # Form progress, presets, live burden checks, and browser drafts
│
├── Training/
│   └── Loan_Prediction_using_ML.ipynb   # Jupyter notebook detailing data analysis and model comparison
│
├── requirements.txt                 # Required Python package dependencies
└── runtime.txt                      # Target Python environment runtime spec (python-3.11.9)
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Smart Lender - Loan Eligibility Prediction
Login Credentials (Demo)	**NA** (Publicly accessible interface; does not require administrative login credentials to request a prediction)
Platform / Hosting Provider	Render
Repository Link	https://github.com/Geethika-27/Smart-Lender/
Demo Video Link	https://youtu.be/EBul2zKoNv4

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:
Follow these step-by-step instructions to set up and run the application in a local development environment:

Prerequisites
Python 3.8+ installed (recommended version: Python 3.11.9)
Visual Studio Code or command-line terminal access
Local Running Procedure
Extract and Navigate to Project Directory:
cd SmartLender
Initialize a Virtual Environment (Optional but Recommended):
python -m venv venv
Activate on Windows:
venv\Scripts\activate
Activate on macOS/Linux:
source venv/bin/activate
Install Dependencies: Install requirements using the project package file:
pip install -r requirements.txt
Verify / Recreate Trained Model (Optional): Pre-trained serialized model and scaling instances are pre-packaged in the workspace. To train the models from scratch:

Open Training/Loan_Prediction_using_ML.ipynb in Jupyter Notebook/VS Code and execute all cells. This exports the trained XGBoost model as rdf.pkl and StandardScaler as scale.pkl into the /Flask folder.
Start Flask Server: Navigate into the web application folder and run the boot script:
cd Flask
python app.py
Open in Browser: Open your browser and navigate to:
http://127.0.0.1:5000

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Internal exception details shown to users (information leak).	Return generic user-facing errors and log full details server-side; avoid rendering raw exception text.
2	No Database Persistence: prediction reports, inputs, and confidence bands are processed on the fly and are not stored in any back-end database.	Workaround: Reloading the app resets the forms. Users can leverage the client-side draft functions ("Save Draft" / "Load Draft" buttons in the UI) to retain local configurations in browser localStorage.
3	Input Out-of-Bounds Scaling: Supplying extreme inputs (e.g. $10,000,000 income) outside the training dataset distribution limits may distort target scaling values.	Workaround: Use realistic applicant scenarios. Form fields have been constrained with logical minimum rules (e.g., minimum loan term of 1 day) to prevent system faults.